



The new networking

Work from home is changing enterprise networking, causing bandwidth and security concerns. <https://digit.in/may20-09>



5G conspiracies

Conspiracies related to 5G are causing widespread vandalisation of telecom infrastructure. <https://digit.in/may20-10>

```

}

function oldFunctionResults() {
    // Existing code that works just fine
}

function newFunctionResults() {
    // Add your new function code here
}

```

Explanation:

What we've basically done here is to add a new Feature Flag in the form of a commented conditional statement. The variable useNewFunction is controlling which branch of the code gets executed. It's initialised with a boolean False so that the old code which lies under oldFunctionResults continues to be executed. We assign boolean True to the useNewFunction variable in the next line but it is commented out so it never gets executed. When you want to switch to the new code, simply uncomment it and the code will move to execute the branch under newFunctionResults. While this is a rather simplistic way of implementing a feature flag, there's only one active code branch. You can use conditional statements to send some requests to one function and the rest to another if you want to perform some form of A/B testing.

USAGE SCENARIOS FOR FEATURE FLAGS

Now that you've gotten a gist of how Feature Flags are implemented in code, let's take a look at all the different use cases where it has been proven to be useful.

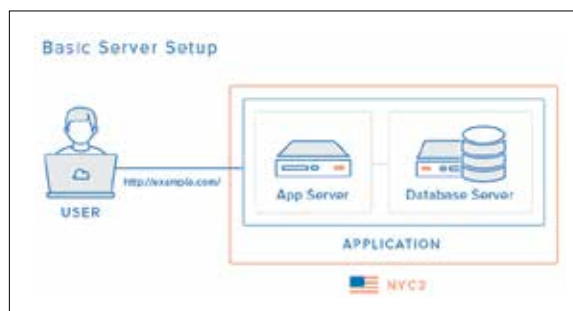
1. Testing new code in production

The obvious use case that we've mentioned multiple times has been to introduce new functionality on the existing production application without breaking functionality. Feature Flags allow software developers to quickly implement and roll back new features

without having to go through the entire routine of restoring a backup. On productions where the code base as well as the database tend to be massive, each operation of pushing code and reverting can affect your uptime and that's never a metric you want to mess with. This does inherently pose a risk of something going wrong since you are testing on the production server so your Feature Toggles have to be implemented in such a manner that you do not end up messing the production application. As an example, you might want to switch off caching if you're working on a web application because certain elements of the code will be pulled from the older cache and might not work properly with the new code that's implemented via a Feature Flag.

2. Canary Deployment

Canary deployment is when you test your new code on a subset of your overall user base. This is especially helpful when you're implementing a beta programme. Users who are part of the beta programme will have their user IDs flagged and whenever your Feature Flag has to process a request from a beta user, they will be diverted into the new code while all your normal users will continue to go via the existing functional code. This also

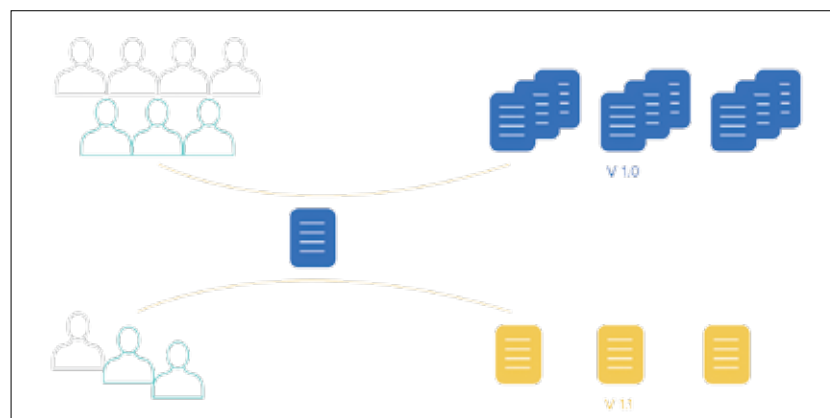


Feature flags are used to test new code

allows you to segregate your users without consent. Let's say the law of the land in a certain country requires you to make certain provisions in your application. Using feature flags, you can route these users originating from that particular country through a different branch in the code.

3. A/B Testing

A/B Testing is more or less like the Canary deployment use case. When you wish to introduce a new function into your application, you would want to gauge the efficacy of this new change before implementing it for all users. In such cases, you would send some users through one code brand and some users via another code branch. Unlike Canary deployment, you do not specifically target users in A/B testing since you want to avoid introducing any biases into your test. A/B testing is especially useful for testing out new features in web applications. Let's say you have a ticketing website where users can find entertainment shows and buy



Canary deployment to a subset user base